

PROJECT DOCUMENTATION

Blood Bank Management System

MERN Stack on Kubernetes with CI/CD and Monitoring

Author: Sarang Sunil Bhagne

Role: DevOps and Cloud Engineer (Fresher)

Year: 2025

GitHub: github.com/Sheren-04/Blood-Donation_Management-System

Live URL: <https://frontend-service-sheren04-dev.apps.rm2.thpm.p1.openshiftapps.com>

1. PROJECT OVERVIEW

The Blood Bank Management System (BBMS) is a full-stack web application built on the MERN stack — MongoDB, Express.js, Node.js, and React. The application manages blood donation operations including donor registration, blood inventory tracking, blood requests, and admin management.

What began as a simple MERN application deployed on a single EC2 instance was evolved into a production-grade Kubernetes deployment on Red Hat OpenShift, complete with automated CI/CD pipelines using GitHub Actions and a monitoring stack using Prometheus and Grafana.

Application Modules

- **Admin Panel:** Secure login, dashboard, and full control over all modules.
- **Blood Inventory:** Tracks all 8 blood groups with unit availability, price per unit, and status (Available, Low, Out of Stock).
- **Blood Requests:** Patients can submit requests with details like blood group, units, urgency level, delivery method, and payment mode.
- **Donor Management:** Register and manage blood donors with availability tracking and last donation date.
- **Contact Messages:** Public contact form with admin reply and status management.

2. TECH STACK

Application

React	Node.js	Express.js	MongoDB Atlas	TypeScript
-------	---------	------------	---------------	------------

Infrastructure and DevOps

Docker	Kubernetes	OpenShift	GitHub Actions	Terraform
--------	------------	-----------	----------------	-----------

Monitoring

Prometheus	Grafana	AWS CloudWatch
------------	---------	----------------

Supporting Tools

Nginx	Ansible	Docker Hub	MongoDB Atlas	Git
-------	---------	------------	---------------	-----

Category	Technology	Purpose
Frontend	React + TypeScript + Vite	User interface and client-side routing
Backend	Node.js + Express.js + TypeScript	REST API and business logic
Database	MongoDB Atlas (Cloud)	Persistent cloud database
Containerization	Docker + Docker Compose	Image building and local orchestration
Orchestration	Kubernetes on OpenShift	Production container management
CI/CD	GitHub Actions	Automated build and deploy pipeline
Reverse Proxy	Nginx	API routing, static file serving
IaC	Terraform + Ansible	Infrastructure and config automation
Monitoring	Prometheus + Grafana	Metrics collection and dashboards
Image Registry	Docker Hub	Central Docker image storage

3. ARCHITECTURE AND DESIGN

3.1 Phase 1 — Initial EC2 Deployment

The application was first deployed on a single AWS EC2 instance to validate that Docker images worked correctly end to end.

- Both frontend and backend ran as Docker containers on one EC2 instance.
- A shared Docker network (bloodbank_network) connected the two containers internally.
- Nginx inside the frontend container proxied /api requests to the backend container on port 5000.
- MongoDB Atlas served as the cloud database — no local database container was needed.
- The backend received environment variables via a backend.env file on the server.
- Frontend was exposed on port 80 (clean URL, no port in browser) via Nginx.

3.2 Phase 2 — Kubernetes on OpenShift

After successful EC2 testing, the entire application was migrated to a Kubernetes cluster running on Red Hat OpenShift Sandbox.

- **Namespace:** sheren04-dev (allocated by OpenShift Sandbox)
- **Frontend Deployment:** 2 replica pods running React/Nginx image pulled from Docker Hub.
- **Backend Deployment:** 2 replica pods running Node.js image pulled from Docker Hub.
- **Services:** Both frontend and backend exposed via ClusterIP services — internal only.
- **Ingress/Routes:** OpenShift Routes with TLS edge termination handle external HTTPS traffic.
- **ConfigMap:** Stores non-sensitive environment variables like PORT, NODE_ENV, FRONTEND_URL.
- **Secrets:** Stores base64-encoded MONGO_URI and JWT_SECRET.
- **Monitoring:** Prometheus and Grafana deployed as separate pods in the same namespace.

3.3 Traffic Flow

- **Browser** sends HTTPS request to OpenShift Route URL.
- **OpenShift Router** terminates TLS at the edge and routes to the appropriate service.
- **Frontend Service (ClusterIP)** receives / traffic and distributes across 2 frontend pods.
- **Backend Service (ClusterIP)** receives /api traffic (proxied by Nginx inside frontend pod) and distributes to 2 backend pods.
- **Backend pods** connect to MongoDB Atlas over the internet using the connection string from Kubernetes Secret.

3.4 Kubernetes Folder Structure

- k8s/config/configmap.yml — environment variables
- k8s/config/secret.yml — sensitive credentials
- k8s/config/nginx-configmap.yml — custom Nginx configuration
- k8s/backend/deployment.yml — Node.js deployment
- k8s/backend/service.yml — backend ClusterIP service
- k8s/frontend/deployment.yml — React/Nginx deployment with nginx ConfigMap mount
- k8s/frontend/service.yml — frontend ClusterIP service
- k8s/ingress/ingress.yml — ingress with TLS
- k8s/monitoring/prometheus/ — Prometheus deployment, service, configmap, RBAC
- k8s/monitoring/grafana/ — Grafana deployment and service

4. FEATURES AND FUNCTIONALITY

4.1 Application Features

Feature	Description
Admin Authentication	JWT-based secure login with bcrypt password hashing. Session stored in sessionStorage.
Blood Inventory	Tracks 8 blood groups with real-time unit count, price per unit, and status — auto-seeded on first run.
Blood Requests	Full request lifecycle — submit, view all requests, update delivery status, delete. Includes urgency and payment mode.
Donor Management	Register donors with blood group, age, city, and availability. Filter and manage donor list.
Contact Messages	Public contact form with subject and phone. Admin can view, reply, and update status.
API Proxy via Nginx	Frontend Nginx proxies /api/* to backend service internally — no CORS issues in production.
Auto Admin Seed	A createAdmin script bootstraps the first admin account directly into MongoDB on first deploy.

4.2 DevOps and Infrastructure Features

Feature	Description
Docker Multistage Build	Separate build and production stages — keeps final images lean by excluding build-time dependencies.
Kubernetes Rolling Updates	Zero-downtime deployments — new pods are created and verified before old pods are terminated.
TLS Edge Termination	HTTPS enforced via OpenShift Edge Routes — SSL handled at the router, not in the app.
ConfigMap Volume Mount	Nginx config injected as a volume mount — allows runtime config changes without rebuilding the image.
Health Probes	Liveness and readiness probes on all pods — Kubernetes auto-restarts unhealthy containers.
CI/CD Automation	GitHub Actions auto-builds and deploys on every push to main — no manual steps required.
Image Tagging with SHA	Each Docker image tagged with git commit SHA for full version traceability across deployments.
Prometheus Monitoring	Scrapes metrics from Prometheus itself and Kubernetes pods. Node Exporter and kube-state-metrics for cluster health.
Grafana Dashboards	Pre-configured data source connecting to Prometheus for real-time CPU, memory, and pod status dashboards.

5. CHALLENGES AND SOLUTIONS

Every challenge below was encountered for real during the build — each one taught a new layer of depth.

Challenge 1 — MongoDB Authentication Failure

Error	bad auth: authentication failed / querySrv ENOTFOUND _mongodb._tcp.2005
-------	---

- **Root Cause:** The MONGO_URI in backend.env was incorrectly set to just a port number (2005) instead of the full Atlas connection string. Additionally, the URI was missing the database name after the cluster hostname.
- **Solution:** Corrected the URI to include the full mongodb+srv:// format with username, password, cluster hostname, and /bloodbank as the database name. Fixed the password encoding for special characters.
- **Learning:** Always specify the database name explicitly in the Atlas URI. Special characters in passwords must be URL-encoded or avoided entirely.

Challenge 2 — Inventory Seed Validation Failure on Startup

Error	Server startup failed: Inventory validation failed: status: Path status is required.
-------	--

- **Root Cause:** The backend startup script tried to seed inventory data but the Mongoose schema required the status field, which was missing from the seed document. The app connected to the test database instead of bloodbank due to a missing database name in the URI.
- **Solution:** Created a comprehensive MongoDB Atlas setup script that manually created all 5 collections (admins, bloodrequests, contactmessages, donors, inventories) with proper schema validation and seeded all 8 blood groups with correct status values. Fixed the URI to point to the correct database.
- **Learning:** When a backend runs initialization or seed logic on startup, the database must be pre-populated with valid data or the startup itself will fail. Always test connection strings point to the intended database.

Challenge 3 — TypeScript Build Error in CI/CD

Error	error TS7030: Not all code paths return a value (requestController.ts line 134)
-------	---

- **Root Cause:** The updateBloodRequestStatus controller used return res.json() in some code paths and plain res.json() in others, creating inconsistent return types that TypeScript rejected during the docker build stage.
- **Solution:** Added Promise<void> as the return type for the async controller function and standardized all response paths to use res.json() followed by a standalone return statement — consistent with the pattern used in all other controllers.
- **Learning:** In TypeScript Express controllers, always declare return type as Promise<void> and separate the response call from the return statement to avoid type inference errors across code paths.

Challenge 4 — Frontend CrashLoopBackOff on OpenShift

Error	mkdir() /var/cache/nginx/client_temp failed (13: Permission denied)
-------	---

- **Root Cause:** OpenShift enforces strict non-root security context constraints (SCC). The stock Nginx Docker image runs as root and writes temp files to /var/cache/nginx, which OpenShift blocks entirely. Additionally, our nginx.conf listened on port 80 which also requires root privileges.

- **Solution:** Overrode the Nginx configuration via a Kubernetes ConfigMap volume mount, changing the listen port to 8080 and redirecting all temp paths (client_temp, proxy_temp, fastcgi_temp, etc.) to /tmp which is writable by non-root users. Added pid /tmp/nginx.pid as well. This approach avoided rebuilding the Docker image entirely.
- **Learning:** OpenShift SCC is stricter than standard Kubernetes. Any image that assumes root access will fail. ConfigMap volume mounts are a powerful pattern for injecting configuration overrides at runtime without touching the image.

Challenge 5 — App Not Accessible on Other Devices

Issue

Website worked locally but showed Application is not available on mobile and external devices.

- **Root Cause:** The OpenShift Route was created without TLS termination. The plain HTTP route worked on local networks where HTTP is not blocked, but was rejected by mobile browsers and stricter networks that enforce HTTPS.
- **Solution:** Deleted the existing plain HTTP routes and created new OpenShift Edge Routes with TLS edge termination for both the frontend (/) and backend (/api) using `oc create route edge`. OpenShift automatically provisions a trusted certificate.
- **Learning:** In production, always enforce HTTPS. OpenShift Edge Routes make this trivial — one command provisions both the route and the TLS certificate without any certificate management on our end.

Challenge 6 — Minikube Startup Failures

Error

healthz check failed / mkdir /var/cache/nginx: Permission denied / connection refused :8443

- **Root Cause:** The development machine had only 3.8GB of RAM. Minikube was allocated 3072MB by default, leaving less than 800MB for the operating system. This caused the Kubernetes API server to fail its health checks and prevented addons like ingress-nginx from starting.
- **Solution:** Deleted the existing Minikube cluster and restarted with `--memory=2200mb --cpus=2`. This left enough RAM for the OS while giving Minikube sufficient resources to start the API server and bootstrap all components cleanly.
- **Learning:** Minikube is resource-sensitive. Always leave at least 1.5GB free for the host OS. The healthz endpoint at localhost:8443 is the first indicator of API server health — check it before debugging addons.

Challenge 7 — ImagePullBackOff on Kubernetes

Error

ImagePullBackOff / ErrImagePull — pods could not start

- **Root Cause:** Docker images built locally on the development machine existed only in the host Docker daemon. Kubernetes (Minikube) runs its own isolated Docker environment and could not access images from the host machine.
- **Solution:** Rather than loading images manually with `minikube image load`, the cleaner approach was to push all images to Docker Hub and set `imagePullPolicy: Always` in the deployment manifests. This made deployments consistent across local Minikube and production OpenShift.
- **Learning:** Kubernetes always pulls images from a registry. Local Docker images are invisible to the cluster unless explicitly loaded. Using Docker Hub as the central registry is the correct production pattern.

Challenge 8 — Docker Hub Push 401 Unauthorized in GitHub Actions

Error

failed to authorize: 401 Unauthorized — access token has insufficient scopes

- **Root Cause:** The `DOCKER_PASSWORD` GitHub Secret was set to the Docker Hub account password. Docker Hub now requires Personal Access Tokens with explicit Read and Write permissions for programmatic push operations.

- **Solution:** Generated a new Docker Hub Personal Access Token with Read and Write permissions and updated the DOCKER_PASSWORD GitHub Secret with this token. The pipeline authenticated and pushed successfully on the next run.
- **Learning:** Docker Hub and most modern registries have moved away from password authentication for API usage. Always use Access Tokens with the minimum required scopes for CI/CD integrations.

Challenge 9 — Grafana Dashboard Showing N/A

Issue

All Grafana panels showed N/A or No data despite Prometheus being deployed.

- **Root Cause:** Cluster-level metric exporters were not deployed. Grafana dashboards for Kubernetes cluster health require node-exporter (for node CPU/memory metrics) and kube-state-metrics (for pod and deployment state). Without these, Prometheus has no data to serve and all panels return N/A. Additionally, the backend had no /metrics endpoint for Prometheus to scrape.
- **Solution:** Deployed node-exporter as a DaemonSet and kube-state-metrics as a Deployment in the monitoring namespace. Updated the Prometheus ConfigMap to include scrape jobs for both. Added prom-client to the Node.js backend to expose a /metrics endpoint.
- **Learning:** Prometheus is a pull-based system — it only has data if targets are actively exposing metrics. Kubernetes cluster monitoring requires three components working together: Prometheus (scraper), node-exporter (node metrics), and kube-state-metrics (Kubernetes object metrics).

6. CONTRIBUTION

This project was a solo end-to-end DevOps implementation. Every component from database schema design to Kubernetes manifest creation to CI/CD pipeline configuration was designed, built, debugged, and deployed independently.

What Was Done

- **Dockerized** React frontend and Node.js backend with multistage Dockerfiles and pushed production images to Docker Hub.
- **Provisioned** complete MongoDB Atlas database schema by writing and executing a custom JS setup script covering all 5 collections with proper validation rules and seed data.
- **Deployed** and validated the full stack on AWS EC2 first to confirm image correctness before moving to Kubernetes.
- **Authored** all 9 Kubernetes manifest files from scratch — ConfigMap, Secret, nginx ConfigMap, frontend Deployment, backend Deployment, two Services, Ingress, and monitoring manifests.
- **Debugged** and resolved OpenShift-specific security constraints by overriding Nginx configuration via ConfigMap volume mount without rebuilding images.
- **Configured** TLS edge termination via OpenShift Routes to enable HTTPS access from all networks and devices.
- **Built** separate GitHub Actions CI/CD pipelines for frontend and backend — each automatically builds, tags with git SHA, pushes to Docker Hub, and triggers a rolling deployment on OpenShift.
- **Deployed** Prometheus with custom ConfigMap scrape configuration and Grafana with preconfigured Prometheus data source for monitoring.
- **Diagnosed** and documented every real error encountered with its root cause and resolution.

7. RESULTS AND IMPACT

Metric	Outcome
Deployment	Live on OpenShift Kubernetes — accessible via public HTTPS URL from any device or network.
Availability	2 replica pods each for frontend and backend — automatic recovery if any pod fails.
Downtime on Update	Zero — GitHub Actions triggers rolling updates where new pods are verified before old ones terminate.
Deployment Effort	Zero manual steps — every push to main branch automatically builds, pushes, and deploys.
Security	HTTPS enforced on all routes. Credentials stored in Kubernetes Secrets, never in code or image.
Monitoring	Prometheus and Grafana live — scraping pod and cluster metrics in real time.
Database	MongoDB Atlas cloud database with network access policies and secure connection via K8s Secret.
Image Versioning	Every deployment tagged with git commit SHA — full audit trail of what code runs in production.

Before vs After

Aspect	Before (EC2)	After (Kubernetes)
Infrastructure	Single EC2 instance	OpenShift Kubernetes cluster
Replicas	1 container each	2 pods each (frontend and backend)
Deployment	Manual SSH and Docker run	Automated via GitHub Actions
Downtime on deploy	Yes — stop then start	Zero — rolling update
HTTPS	Not configured	TLS edge termination enforced
Monitoring	None	Prometheus and Grafana stack
Secret management	Plain .env file on server	Kubernetes Secrets (base64 encoded)
Accessibility	Local network only	Public HTTPS, any device, any network

8. LEARNINGS

8.1 Kubernetes and OpenShift

- Kubernetes core primitives — how Deployments, ReplicaSets, Pods, Services, Ingress, ConfigMaps, and Secrets interact to deliver a resilient production application.
- OpenShift Security Context Constraints (SCC) are stricter than standard Kubernetes. Any container assuming root access will fail. Non-root compliant images and writable path redirection are essential.
- ConfigMap volume mounts are a powerful runtime configuration injection mechanism that decouples configuration from the image build process entirely.
- ClusterIP Services with Ingress/Routes is the correct production pattern — not LoadBalancer per service, which is expensive and unnecessary.
- Rolling updates guarantee zero downtime by creating new pods, verifying readiness, then terminating old ones — this is Kubernetes default deployment strategy.
- Liveness and readiness probes are critical — without them Kubernetes cannot distinguish a starting pod from a broken one.

8.2 CI/CD with GitHub Actions

- GitHub Actions with docker/build-push-action is more efficient than separate docker build and docker push commands — it handles layer caching automatically.
- Tagging images with git SHA (`SHA`) provides complete deployment traceability — every running pod can be traced back to an exact commit.
- path filters (paths: backend/**) prevent unnecessary pipeline triggers — frontend changes should not rebuild the backend image.
- `oc set image` is the correct Kubernetes update command — it patches only the image field of the deployment spec without re-applying the entire manifest.
- Docker Hub now requires Personal Access Tokens, not account passwords, for CI/CD authentication.

8.3 Debugging and Problem Solving

- `oc describe pod` and `oc logs` are the two most important debugging tools in Kubernetes — always check these before anything else.
- `ImagePullBackOff` almost always means either the image does not exist in the registry, the tag is wrong, or authentication to the registry failed.
- `CrashLoopBackOff` means the container starts and then crashes repeatedly — always check logs immediately after the first crash before it gets cleared.
- Port-forwarding in OpenShift Sandbox times out due to shared cluster restrictions — use `oc expose service` instead to get a route for testing.
- Prometheus N/A on all panels is not a Grafana issue — it means the data source has no data, which means exporters are not deployed or targets are not scraping.

8.4 Infrastructure and Architecture

- Always test Docker images on a VM (EC2) before deploying to Kubernetes — this confirms the image itself works before adding orchestration complexity.
- MongoDB Atlas Network Access must allow 0.0.0.0/0 for dynamic cluster environments like OpenShift Sandbox where pod IPs are not predictable.
- Separating frontend and backend into independent pods is the correct microservices pattern — it allows independent scaling, independent failure domains, and independent deployments.

9. LINKS AND REFERENCES

Project Repositories

- **Blood Bank Management System (Main App):** github.com/Sheren-04/Blood-Donation_Management-System
- **Self-Healing AWS Deployment:** github.com/Sheren-04/self-healing-webapp-deployment

Live Deployment

- **Frontend (OpenShift):** <https://frontend-service-sheren04-dev.apps.rm2.thpm.p1.openshiftapps.com>
- **Grafana Dashboard:** <http://grafana-service-sheren04-dev.apps.rm2.thpm.p1.openshiftapps.com>
- **Prometheus:** <http://prometheus-service-sheren04-dev.apps.rm2.thpm.p1.openshiftapps.com>

Author Profiles

- **GitHub:** github.com/Sheren-04
- **LinkedIn:** linkedin.com/in/sarangbhagne
- **Email:** sarangbhagne80@gmail.com

Key Tools and Documentation

- **Kubernetes Documentation:** kubernetes.io/docs
- **OpenShift Sandbox:** developers.redhat.com/developer-sandbox
- **GitHub Actions Documentation:** docs.github.com/en/actions
- **Prometheus Documentation:** prometheus.io/docs
- **Docker Documentation:** docs.docker.com
- **MongoDB Atlas:** mongodb.com/atlas